45

BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

14. İlke: Gerçeği Testler Taşır (AIUP Test Piramidi)

Principle 14: Tests Carry the Truth — The AIUP Test Pyramid

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              AIUP TEST PİRAMİDİ VE YENİDEN ÜRETİM KORUMASI   │
└──────────────────────────────────────────────────────────────┘

          ▲
         / \
        /   \      UÇTAN UCA YOLCULUK TESTLERİ (E2E Journey Tests)
       / E2E \     - Kaynak: TC-XXX Dokümanları (docs/test_cases/)
      /------- \   - Kapsam: Birden fazla use case'in ardışık zinciri
     /   INTE-  \
    /   GRATION  \ ENTEGRASYON TESTLERİ (Integration / View Tests)
   /--------------\- Kaynak: UC Şartnamesi (Gerçek UI & Veritabanı ile)
  /      UNIT /    \
 /     VIEW TESTS   \ BİRİM TESTLERİ (Unit Tests per Flow)
/--------------------\- Kaynak: Her alternatif akış (A1, A2) ve BR kuralları!

🎯 İLKE 14'ÜN ÖZÜ:
"Kod her an silinip baştan üretilebilir. Kodu güvenle yeniden üretilebilir
 kılan tek şey DIŞARIDAN onu sabitleyen TESTLERDİR!"
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Lunaparktaki hız trenini (rollercoaster) düşünün: Trenin vagonunu istediğiniz renge boyayabilir, koltuklarını değiştirebilirsiniz. Ama vagonun raydan çıkmasını engelleyen şey altındaki çelik kilitlerdir! Testler o çelik kilitlerdir. AI kodu baştan üretse bile testler trenin raydan çıkıp uçuruma yuvarlanmasını engeller!

🇬🇧 ENGLISH (ELI5)

Think of a roller coaster: You can repaint the coaster cars or upgrade the upholstery anytime. But what keeps the car from flying off the tracks at 100 mph are the steel under-friction safety wheels! Automated tests are those steel clamps. Even if an AI completely regenerates the engine, the tests guarantee the car stays on track.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

14. İlke: Gerçeği Testler Taşır (Tests Carry the Truth): Use-Case 3.0'daki en çarpıcı cümle şudur: 'Test senaryoları, bir use case'in anlatısından bile daha önemli olan en kritik parçasıdır; çünkü use case'in ne zaman başarıyla tamamlandığını sadece testler tanımlar!' AI Unified Process bu tavsiyeyi yapısal bir zorunluluğa dönüştürür. 4. İlke 'Test-Protected' (Testle Korunan): Kod her an baştan üretilebilen geçici bir derleme çıktısıdır. Eğer bir sistemin davranışı kodun dışında bir mekanizma tarafından sabitlenmemişse, o sistemi AI ile yeniden üretmek kumar oynamaktır! Test piramidi doğrudan şartname zincirine bağlanır: 1) Birim/Akış Testleri: Şartnamedeki her bir doğrulama adımını ve her bir alternatif akışı (A1, A2) tek tek sınar. 2) Entegrasyon Testleri: Bir use case'in tamamını gerçek arayüz ve servislerle test eder. 3) Yolculuk Testleri (TC-XXX): Birden fazla use case'i bağlayan iş süreçlerini test eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 14: Tests Carry the Truth & The AIUP Test Pyramid: Use-Case 3.0 asserts boldly: 'The test cases are the most important part of a use case's description, more important even than the narrative.' In the AI Unified Process, this transitions from agile guidance to absolute architectural necessity. Principle 4 (Test-Protected) establishes that because code can be generated, refactored, or discarded at will, intent must be pinned by artifacts outside the code. The tests freeze observable behavior. The test hierarchy aligns cleanly with the artifact chain: 1) Unit/Flow Tests: Derived directly from the use case validations and alternative flows (BR-XXX). 2) Integration Tests: Drive an entire use case through the presentation layer. 3) Journey Tests (TC-XXX): Verify multi-use-case lifecycles where state transitions cross use case boundaries.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Alternatif Akışları Doğrulayan Test-Protected Mimari
package com.maang.aiup.testing;

import java.util.Optional;

public class TestCarriesTruthDemo {
    public record BorrowResult(boolean success, String message) {}

    public static class BorrowValidator {
        public static BorrowResult validate(int availableCopies, int currentLoans, boolean hasOverdue)
{
            // BR-004: Kopya kontrolü -> Alternatif A1
            if (availableCopies <= 0) return new BorrowResult(false, "A1: Kopya kalmadı.");
            // BR-005: Limit kontrolü -> Alternatif A2
            if (currentLoans >= 5) return new BorrowResult(false, "A2: Limit aşıldı.");
            // BR-006: Gecikmiş borç -> Alternatif A3
            if (hasOverdue) return new BorrowResult(false, "A3: Gecikmiş borç var.");

            return new BorrowResult(true, "Ödünç başarılı.");
        }
    }

    // TEST MOTORU (Şartnamenin bekçisi)
    void main() {
        // Test 1: A1 Akışını sına
        var resA1 = BorrowValidator.validate(0, 1, false);
        assert !resA1.success() && resA1.message().contains("A1") : "A1 Testi Başarısız!";

        // Test 2: A2 Akışını sına
        var resA2 = BorrowValidator.validate(2, 5, false);
        assert !resA2.success() && resA2.message().contains("A2") : "A2 Testi Başarısız!";

        // Test 3: Başarı Yolunu sına
        var resOk = BorrowValidator.validate(2, 2, false);
        assert resOk.success() : "Mutlu Yol Testi Başarısız!";

        System.out.println("✅ Tüm akış testleri geçti! Şartname koruma altındadır.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Testi yazılmamış bir use case şartnamesi yalnızca bir temenniden ibarettir! Her alternatif akışın en az bir otomatik testi olmak zorundadır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

A use case without automated test cases is merely a wish. Every alternative flow must have at least one automated assertion guarding it against regression.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kod geçicidir, test kalıcıdır: Sistemi güvenle yeniden üretilebilir kılan testlerdir.

• Şartnamedeki her bir doğrulama (validates) ve alternatif akış (A1, A2) için test yazılır.

• Yolculuk testleri (TC-XXX) use case'ler arasındaki durum ve veri akışını denetler.

🎯 Key Architectural Takeaways

• Code is ephemeral; tests are enduring. Automated tests make regeneration safe.

• Every validation step and alternative flow in the spec maps to a specific test case.

• Journey tests (TC-XXX) verify cross-use-case boundaries and continuous state progression.